



# **Comparative Study of Machine Learning and Deep Learning Approaches for Mental Health Detection from Social Media Text**

## **PROJECT SUPERVISOR**

Ms. Fizza Aqeel

## **PROJECT TEAM**

|                         |          |
|-------------------------|----------|
| Muhammad Tahir Haider   | 22K-6002 |
| Muhammad Huzaifa Saleem | 22K-5103 |
| Syed Muhammad Sufyan    | 22K-6012 |

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in  
Computer Science.

FAST SCHOOL OF COMPUTING  
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES  
KARACHI CAMPUS  
Spring 2026

|                    |                   |          |
|--------------------|-------------------|----------|
| Project Supervisor | Ms. Fizza Aqeel   |          |
| Project Team       | M. Tahir Haider   | K22-6002 |
|                    | M. Huzaifa Saleem | K22-5103 |
|                    | Syed M. Sufyan    | K22-6012 |
| Submission Date    | May 11, 2026      |          |

**Ms. Fizza Aqeel** \_\_\_\_\_  
**Supervisor**

FAST SCHOOL OF COMPUTING  
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES  
KARACHI CAMPUS



# AbSTRACT

Mental health disorders such as depression and suicidal ideation represent a growing global public health crisis, affecting hundreds of millions of people worldwide. Traditional clinical diagnostic methods face significant challenges including social stigma, delayed reporting, and limited scalability. The pervasive use of social media platforms such as Twitter and Reddit have created an unprecedented opportunity to harness user-generated content for the automated, early detection of mental health conditions through Natural Language Processing (NLP) and machine learning techniques.

This project presents a rigorous comparative study of classical Machine Learning (ML) models — Logistic Regression (LR), Support Vector Machine (SVM), Naïve Bayes (NB), and Random Forest (RF) — against advanced Deep Learning (DL) architectures — Convolutional Neural Networks (CNN), Long Short-Term Memory networks (LSTM), and Bidirectional LSTM (BiLSTM) — and Transformers (BERT) for binary sentiment/mental health classification on social media text.

All models are trained and evaluated on two distinct datasets using unified preprocessing pipelines: the Sentiment140 dataset (1.6 million entries) for depression sentiment analysis, and a combined Suicide dataset (~242,000 entries) from two publicly available sources for suicide ideation detection, ensuring fair and reproducible comparison across both domains. Classical models utilize TF-IDF feature representations, while deep learning models employ embedding-based strategies. For depression, deep learning models use a hybrid embedding strategy combining frozen GloVe Twitter 27B embeddings with trainable domain-specific embeddings. For suicide, models use Keras Tokenizer-based embeddings with trainable layers. BERT fine-tuning is applied on both datasets as the transformer-based benchmark. Evaluation is performed using Accuracy, Precision, Recall, F1-Score,

ROC-AUC, and the Precision-Recall curve.

The findings demonstrate clear trade-offs between model families: classical ML models offer computational efficiency and interpretability, deep learning models improve performance through automatic feature learning and sequential context modelling, while transformer-based models such as BERT achieve strong overall performance due to their superior contextual understanding of text. This study provides a reproducible benchmark and critical analysis of the strengths, weaknesses, and practical trade-offs of each approach for computational mental health detection from social media text.

# Table of Contents

|                                                  |    |
|--------------------------------------------------|----|
| 1. Introduction.....                             | 9  |
| 1.1. Project Aim and Objectives .....            | 9  |
| 1.2. Scope and Limitations .....                 | 10 |
| 2. Related Work.....                             | 11 |
| 2.1. Classical Machine Learning Approaches ..... | 11 |
| 2.2. Deep Learning Approaches.....               | 11 |
| 2.3. Transformer-Based Models .....              | 12 |
| 2.4. Existing Comparative Studies .....          | 13 |
| 3. Requirements .....                            | 14 |
| 3.1. Functional Requirements.....                | 14 |
| 3.2. Non-Functional Requirements.....            | 14 |
| 3.3. Dataset Requirements.....                   | 15 |
| 3.4. Development Environment.....                | 15 |
| 4. Design.....                                   | 16 |
| 4.1. System Architecture Overview .....          | 16 |
| 4.2. Preprocessing Pipeline Design .....         | 17 |
| 4.2.1. Text Normalization.....                   | 17 |
| 4.2.2. Feature Engineering Paths .....           | 18 |
| 4.3. Model Architecture Designs .....            | 19 |
| 4.3.1. Classical ML Models — Depression .....    | 19 |
| 4.3.2. Classical ML Models — Suicide.....        | 19 |
| 4.3.3. Deep Learning Models — Depression .....   | 20 |
| 4.3.4. Deep Learning Models — Suicide.....       | 21 |

|                                                     |    |
|-----------------------------------------------------|----|
| 4.4. Evaluation Framework Design .....              | 21 |
| 5. Implementation .....                             | 22 |
| 5.1. Depression Dataset Pipeline .....              | 22 |
| 5.1.1. Data Preprocessing.....                      | 22 |
| 5.1.2. Classical ML Models.....                     | 22 |
| 5.1.3. Deep Learning Models .....                   | 28 |
| 5.1.4. BERT.....                                    | 32 |
| 5.2. Suicide Dataset Pipeline.....                  | 33 |
| 5.2.1. Data Loading, Merging and Preprocessing..... | 33 |
| 5.2.2. Classical ML Models .....                    | 34 |
| 5.2.3. Deep Learning Models.....                    | 39 |
| 5.2.4. BERT .....                                   | 43 |
| 5.3. Colab-Specific Optimizations .....             | 45 |
| 6. Testing and Evaluation.....                      | 46 |
| 6.1. Evaluation Methodology.....                    | 46 |
| 6.2. Results Summary — Depression Dataset .....     | 47 |
| 6.3. Results Summary — Suicide Dataset .....        | 47 |
| 6.4. Classical ML Analysis .....                    | 47 |
| 6.5. Deep Learning Analysis .....                   | 49 |
| 6.6. Key Trade-offs .....                           | 50 |
| 6.7. Testing Strategy .....                         | 51 |
| 7. Conclusion .....                                 | 53 |
| 7.1. Future Work.....                               | 54 |
| 8. References .....                                 | 55 |



# 1. Introduction

Mental health issues, including depression, anxiety, and suicidal ideation, are among the most prevalent and underdiagnosed conditions in modern society. According to the World Health Organization, depression alone affects over 280 million people globally, yet the majority of those affected do not receive timely care due to social stigma, limited access to clinical services, and the subjective nature of self-reporting. Early detection is widely recognized as the most effective strategy for improving outcomes and reducing the societal burden of mental illness.

The proliferation of social media — particularly microblogging platforms such as Twitter and Reddit — has fundamentally altered how individuals express their emotional states and psychological experiences. Users routinely share thoughts, feelings, and personal struggles in real time, creating vast repositories of unstructured text data that contain latent signals of mental health status. This digital footprint presents a compelling opportunity for computational approaches to detect mental health indicators at scale, non-invasively, and proactively.

Natural Language Processing (NLP) and machine learning have emerged as the dominant paradigms for extracting meaningful patterns from social media text. However, the field faces a critical methodological challenge: existing studies are highly heterogeneous in their choice of datasets, preprocessing pipelines, feature representations, and evaluation protocols, making cross-study comparisons unreliable. It remains unclear whether performance differences observed between model families reflect genuine algorithmic superiority or simply artefacts of inconsistent experimental conditions.

## 1.1. Project Aim and Objectives

This project aims to conduct a rigorous comparative evaluation of classical Machine Learning and Deep Learning models for detecting mental health indicators from social media text, using a unified experimental framework to ensure fairness and reproducibility.

The specific objectives are:

- To collect and preprocess large-scale social media datasets through standardized NLP pipelines including tokenization, normalization, and feature extraction for two distinct mental health domains: depression and suicide ideation.

- To implement and evaluate four classical ML models (LR, SVM, NB, RF) using TF-IDF feature representations on both datasets.
- To design and train three deep learning models (CNN, LSTM, BiLSTM) with embedding-based strategies optimized for GPU execution on Google Colab T4.
- To fine-tune BERT (bert-base-uncased) as the transformer-based benchmark on both datasets.
- To evaluate all models using standardized metrics: Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Average Precision (PR-AUC).
- To provide a comparative analysis of trade-offs between model families in terms of performance, interpretability, training efficiency, and computational cost across both mental health domains.

## **1.2. Scope and Limitations**

The scope of this project is strictly limited to academic comparative research on binary text classification for mental health detection. The system is not intended as a clinical tool and has not been clinically validated. All experiments are conducted on publicly available benchmark datasets. The project does not involve real-time monitoring, personalized user profiling, or deployment as a production application.

## **2. Related Work**

Research in this domain spans traditional machine learning models, deep learning architectures, and transformer-based approaches. This section reviews each category and their comparative performance characteristics.

### **2.1. Classical Machine Learning Approaches**

Classical machine learning models remain valuable due to their computational efficiency and high degree of interpretability compared to deep learning approaches, particularly for moderately sized datasets [1].

Support Vector Machine (SVM) is among the most widely reported ML classifiers for text classification tasks, often employing different kernels to effectively separate classes [2], [3], [4], [5]. Hybrid models integrating SVM, such as the NeuroVibeNet framework, have achieved high accuracy in multimodal detection scenarios [6].

Logistic Regression (LR) is particularly favored for its interpretability, as it readily captures positive and negative feature associations relevant to clinical interpretation [1]. When effectively tuned, LR has demonstrated accuracy comparable to SVM in distinguishing depressed users from control groups [7], [8].

Naïve Bayes (NB) is primarily employed as a simple, efficient baseline model due to its probabilistic nature and computational efficiency [3], [5].

Random Forest (RF) is an ensemble method that consistently yields robust performance in depression detection tasks through its use of multiple decision trees [3], [8], [9]. Advanced tree-based models like XGBoost have demonstrated exceptional performance in multi-level severity classification [10].

### **2.2. Deep Learning Approaches**

Deep learning models excel by automatically learning hierarchical, complex feature representations, bypassing the constraints of manual feature engineering required by traditional ML methods [1], [11].

Convolutional Neural Networks (CNN) are employed to capture localized, high-level features and hierarchical representations within social media posts. CNNs have been successfully combined with

linguistic metadata for effective early detection, and hybrid architectures like SBERT-CNN (combining Sentence BERT embeddings with CNN layers) have achieved impressive F1 scores of 0.86 in depression identification [4], [11].

Long Short-Term Memory (LSTM) and Bidirectional LSTM (BiLSTM) networks are fundamental for processing sequential text data. LSTMs effectively capture long-range dependencies, while BiLSTM surpasses standard LSTM performance by processing input context in both forward and backward directions, thereby improving sequential pattern detection [12]. BiLSTM models augmented with attention mechanisms have demonstrated particularly competitive results in text-based depression detection [13].

### **2.3. Transformer-Based Models**

Transformer architectures, particularly BERT (Bidirectional Encoder Representations from Transformers), leverage bidirectional encoding and attention mechanisms to create highly contextualized word representations, establishing state-of-the-art benchmarks for accuracy in complex NLP tasks [7], [12], [14].

BERT's bidirectional attention mechanism enables it to capture contextual nuances critical for mental health detection, such as understanding how negations or qualifiers modify emotional expressions. Unlike traditional models that process text sequentially, BERT considers both left and right context simultaneously, making it particularly effective at understanding subtle linguistic patterns associated with mental health conditions [5], [7], [15].

Fine-tuned variants of BERT demonstrate superior classification performance in depression detection. Domain-specific models like MentalBERT, pre-trained on mental health corpora, have achieved the highest reported F1-scores (97.31%) in multi-level depression severity detection, underscoring the value of specialized pre-training [10]. Alternative variants like RoBERTa and DistilBERT offer performance-efficiency trade-offs suitable for resource-constrained scenarios [5], [7], [15].

To maximize performance, BERT embeddings are often combined with recurrent architectures. The BERT-BiLSTM hybrid, which integrates BERT's contextual understanding with BiLSTM's sequential modeling capabilities, has demonstrated enhanced performance in capturing complete depression-related content, often yielding superior recall and

F1 scores compared to using either architecture alone [12], [13].

## **2.4. Existing Comparative Studies**

Several recent studies have attempted systematic comparisons between modeling approaches, but with varying methodologies and conclusions. Research comparing ML and DL approaches has found that while transformer models achieve highest accuracy, classical ML remains competitive when interpretability or computational resources are prioritized [1], [5], [7]. Studies examining performance trade-offs note that deep learning excels in accuracy while traditional ML offers advantages in efficiency and transparency [1].

However, these comparative studies reveal methodological limitations including inconsistent preprocessing pipelines across model families, evaluation on different datasets or data splits, and predominant focus on binary classification rather than multi-class detection [2], [7]. Furthermore, few studies provide detailed analysis of computational costs alongside performance metrics, limiting practical guidance for deployment decisions [1].

## 3. Requirements

### 3.1. Functional Requirements

The system shall provide the following core functionalities:

- **Text Preprocessing Pipelines:** Two dataset-specific preprocessing pipelines: (a) a light preprocessing pipeline for the Depression/Sentiment140 dataset (URL/mention tokenization, lowercasing, character deduplication, punctuation spacing) applied to all model families; and (b) a self-contained preprocessing pipeline for the Suicide dataset (lowercasing, URL/mention/hashtag removal, punctuation stripping, whitespace normalization) implemented within each model notebook.
- **Feature Extraction:** TF-IDF (unigram + bigram) for classical ML models; Keras Tokenizer-based integer sequences with embedding layers for deep learning models; WordPiece tokenization with contextual embeddings via bert-base-uncased for the transformer model.
- **Model Training and Evaluation:** All eight model implementations (LR, SVM, NB, RF, CNN, LSTM, BiLSTM, BERT) trained on both datasets with identical train/test splits.
- **Standardized Evaluation:** Confusion matrix, classification report, ROC-AUC curve, and Precision-Recall curve generated for every model.
- **Reproducibility:** Fixed random seeds (42) and deterministic data splits across all experiments.

### 3.2. Non-Functional Requirements

- **Performance:** All deep learning models to complete training within the Google Colab T4 GPU session time limits.
- **Memory Efficiency:** Classical ML models implement batch processing and sparse matrix operations to handle large datasets within available RAM constraints.
- **Scalability:** Preprocessing and feature extraction designed to handle datasets of varying sizes (242K–1.6M samples).
- **Portability:** All code compatible with Google Colab environment; minimal local installation requirements.

### 3.3. Dataset Requirements

The project utilizes three publicly available datasets for mental health classification tasks across two domains:

#### Depression Domain — Sentiment140 Dataset:

- 1,600,000 tweets annotated with binary sentiment labels
- Source: Twitter
- Split: 90/10 stratified train/test with random\_state=42

#### Suicide Domain — Combined from two sources:

1. Suicide\_Detection.csv (Kaggle: Suicide Watch dataset) — ~232,000 entries with binary labels (suicide / non-suicide)
  2. suicide\_only\_clean.csv (Kaggle: Reddit Depression & SuicideWatch dataset, depression entries removed) — ~10,000 suicide-labelled entries
- Combined total: ~242,066 entries (126,029 suicide, 116,037 non-suicide)
  - Split: 80/20 stratified train/test with random\_state=42

Using multiple datasets across two mental health domains allows the project to evaluate model performance and generalization across different conditions rather than limiting the analysis to a single domain.

### 3.4. Development Environment

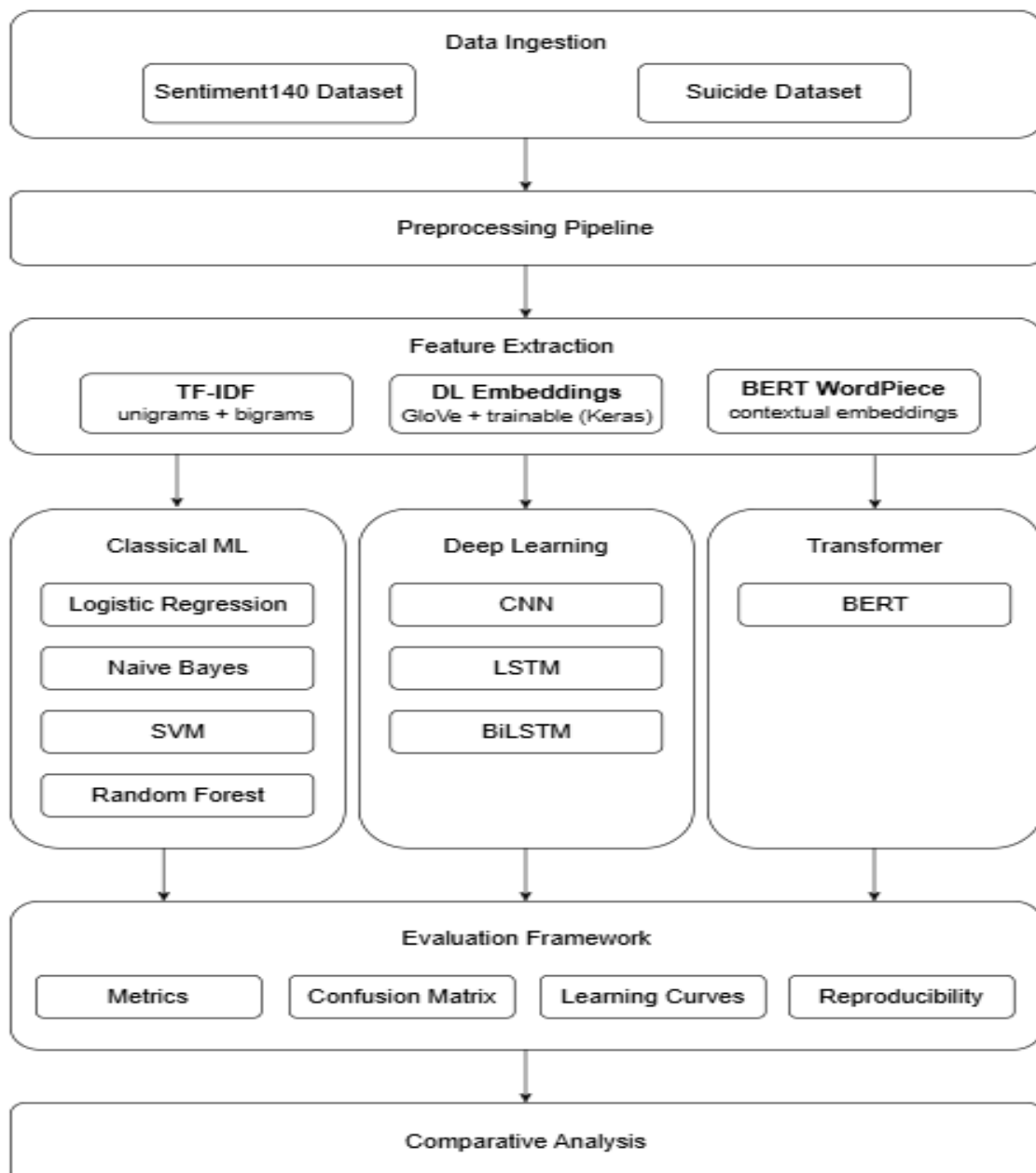
| Component       | Specification                                                                                |
|-----------------|----------------------------------------------------------------------------------------------|
| Platform        | Google Colab (T4 GPU)                                                                        |
| Language        | Python 3.10+                                                                                 |
| ML Framework    | Scikit-learn, TensorFlow 2.x / Keras, PyTorch                                                |
| NLP Libraries   | NLTK, TextBlob, Hugging Face Transformers                                                    |
| Embeddings      | GloVe Twitter 27B 100d (Depression DL models)<br>Keras Embedding 128-dim (Suicide DL models) |
| Visualization   | Matplotlib, Seaborn                                                                          |
| Version Control | Git / GitHub                                                                                 |

## 4. Design

### 4.1. System Architecture Overview

The system follows a modular pipeline architecture designed to ensure consistent, reproducible comparison across all model families and both datasets. The pipeline is divided into five sequential stages: (1) Data Ingestion, (2) Preprocessing and Feature Extraction, (3) Model Training, (4) Evaluation, and (5) Comparative Analysis.

Each dataset follows its own preprocessing path but feeds into the same model architectures with consistent evaluation metrics, enabling meaningful cross-domain comparison.



## **4.2. Preprocessing Pipeline Design**

The preprocessing pipelines were carefully designed to balance the contrasting requirements of classical ML and deep learning models, while adapting to the characteristics of each dataset.

### **4.2.1. Text Normalization**

All models across both datasets share a common foundation of lowercasing and URL/mention handling, but the depth of normalization varies by dataset and model family to suit the characteristics of each text source.

For the Depression dataset, the centralized preprocessing pipeline applies a light normalization strategy designed to preserve as much linguistic signal as possible from the short tweet format. URLs and user mentions are replaced with placeholder tokens ([URL] and [USER] respectively) rather than removed, retaining their positional context within the text. Consecutive repeated characters are reduced to at most two (e.g., "coool" → "cool") to normalize informal spelling while preserving emphasis. Sentiment-bearing punctuation marks (! and ?) are retained and padded with spaces to ensure they are tokenized as distinct units, while all remaining non-alphabetic characters are removed. The BERT model on this dataset applies its own dedicated preprocessing instead, adding HTML entity unescaping and expanding 48 common English contractions (e.g., "won't" → "will not", "I'm" → "I am") via compiled regular expression matching to maximize vocabulary coverage under BERT's WordPiece tokenizer, while retaining all punctuation for BERT to process natively.

For the Suicide dataset, the preprocessing is more aggressive, reflecting the longer Reddit post format (~250 tokens vs. ~50 for tweets) which provides sufficient context even after heavier noise removal. URLs, user mentions, and hashtag symbols are removed entirely rather than replaced with placeholders. All punctuation is stripped using a character translation table, and excess whitespace is collapsed and trimmed. The Suicide BERT model shares the same URL, mention, and hashtag removal steps but omits the punctuation stripping, since BERT's WordPiece tokenizer can meaningfully process punctuation as subword tokens.

One exception to this shared preprocessing pattern is the Random Forest model on the Suicide dataset, which bypasses text normalization and TF-IDF vectorization entirely. Instead, it operates directly on the raw

text and extracts 13 hand-crafted features spanning three categories: structural features (character count, word count, average word length, sentence count, punctuation count, digit count, capital letter ratio), sentiment features (TextBlob polarity and subjectivity, VADER negative/positive/neutral compound scores), and a domain-specific suicide keyword count based on a curated list of 13 crisis-related terms.

#### **4.2.2. Feature Engineering Paths**

Two distinct feature extraction paths are maintained:

- **Classical ML Path:** TF-IDF vectorization producing sparse matrices. For Depression: `ngram_range=(1,2)`, `max_features=100,000`, `min_df=3`, persisted as .npz files. For Suicide: inline TF-IDF vectorization within each model notebook.
- **Deep Learning Path:** Keras Tokenizer with post-padded integer sequences. For Depression: max 200,000 vocabulary, sequence length 50, hybrid GloVe + trainable embeddings. For Suicide: max 20,000 vocabulary, sequence length 250, trainable embeddings only.
- **Transformer Path:** BERT WordPiece tokenization with contextual embeddings via bert-base-uncased.

## 4.3. Model Architecture Designs

### 4.3.1. Classical ML Models — Depression

| Model               | Key Design Choices                                                                                                                                                                                                       | Hyperparameters                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Logistic Regression | Custom implementation from scratch; mini-batch stochastic gradient descent on sparse TF-IDF matrices; numerically stable sigmoid with clipping; binary cross-entropy loss; batched prediction to prevent memory overflow | lr=0.5, batch=5000, iterations=1000                                                                      |
| Naive Bayes         | Multinomial NB with Laplace smoothing; sparse CSR operations throughout; log-sum-exp trick for numerical stability; batched log-probability prediction                                                                   | alpha=0.5 (Laplace smoothing)                                                                            |
| SVM                 | LinearSVC with primal formulation (dual=False) optimized for high-dimensional sparse TF-IDF; Platt scaling (sigmoid) probability calibration on 100,000-sample subset; class_weight='balanced'                           | C=0.1, max_iter=3000                                                                                     |
| Random Forest       | Ensemble of decision trees with warm_start early stopping via OOB error monitoring; stratified sub-sampling (200K) for scalability; Gini-based feature importance extraction; fully parallel training (n_jobs=-1)        | n_estimators=50–300 (early-stopped), max_features='sqrt', max_depth=None, min_samples_leaf=5, patience=3 |

### 4.3.2. Classical ML Models — Suicide

| Model               | Key Design Choices                                                                                                                                                                                                         | Hyperparameters                                                         |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Logistic Regression | Custom implementation from scratch; full-batch gradient descent on sparse TF-IDF matrices; numerically stable sigmoid with clipping; binary cross-entropy loss with L2 regularization; decision threshold tuning (0.3–0.7) | lr=0.5, iterations=1000, lambda=0.01, max_features=20000, threshold=0.5 |
| Naive Bayes         | Multinomial NB built from scratch; self-contained pipeline with integrated preprocessing; Laplace-smoothed log-probabilities; TF-IDF feature extraction (max 20K features)                                                 | alpha=1.0 (Laplace smoothing), max_features=20000                       |

|               |                                                                                                                                                                                                               |                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| SVM           | Custom Linear SVM from scratch; hinge loss with sub gradient descent on sparse TF-IDF; minority class up sampling for class balancing; mini-batch training with support vector identification                 | lr=0.5, lambda=0.00001, batch=10000, iterations=1000          |
| Random Forest | Fully custom implementation from scratch (decision trees + ensemble); Gini impurity splitting; hand-crafted features (13 engineered: structural, sentiment, keyword); bootstrap sampling with majority voting | n_trees=15, max_depth=6, min_size=20, features_per_split=sqrt |

### 4.3.3. Deep Learning Models — Depression

All three deep learning models for Depression share a common hybrid embedding architecture. The design uses a shared input layer feeding into two parallel embedding branches: (1) a frozen GloVe Twitter 100d embedding capturing pre-trained Twitter semantics, and (2) a fully trainable 200-dimensional domain-specific embedding. The outputs are concatenated and passed to a Dropout layer before the model-specific architecture.

| Model  | Architecture                                                                                                                                                                                                                                | Key Parameters                                                                              |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| CNN    | Hybrid embeddings (frozen GloVe 100d + trainable 200d) → Concat → Dropout(0.4) → Conv1D(128, k=5, ReLU) → GlobalMaxPool → Dense(64, ReLU) → Dropout(0.4) → Softmax                                                                          | filters=128, kernel=5, batch=512, epochs=15, lr=1e-3                                        |
| LSTM   | Hybrid embeddings (frozen GloVe 100d + trainable 200d) → Concat → Dropout(0.4) → LSTM(256, recurrent dropout=0.1) → Dense(64, ReLU) → Dropout(0.4) → Softmax                                                                                | units=256, batch=512, epochs=15, lr=1e-3                                                    |
| BiLSTM | Hybrid embeddings (frozen GloVe 100d + trainable 200d) → Concat → Dropout(0.4) → Bidirectional(LSTM(256, recurrent dropout=0.1)) → Dense(64, ReLU) → Dropout(0.4) → Softmax                                                                 | units=256 (512 effective), batch=512, epochs=15, lr=1e-3                                    |
| BERT   | bert-base-uncased with custom [URL]/[USER] tokens; contraction expansion; [CLS] pooler → Dropout(0.1) → Linear classification head; AdamW with linear warmup scheduling; mixed-precision (AMP) training; label smoothing; gradient clipping | max_len=48, batch=128, epochs=5, lr=2e-5, warmup=6%, weight_decay=0.01, label_smoothing=0.1 |

### 4.3.4. Deep Learning Models — Suicide

The suicide deep learning models use a simpler embedding architecture with Keras Tokenizer embeddings, suitable for the Reddit-based text which differs stylistically from tweets.

| Model  | Architecture                                                                                                                                                                                     | Key Parameters                                                                  |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| CNN    | Embedding(20000, 128) → SpatialDropout1D(0.3) → Conv1D(128, k=5, ReLU) → GlobalMaxPool → Dense(128, ReLU) → Dropout(0.5) → Sigmoid                                                               | max_words=20000, max_len=250, batch=64, epochs=10                               |
| LSTM   | Embedding(20000, 128) → SpatialDropout1D(0.2) → LSTM(64, dropout=0.2, recurrent_dropout=0) → Dense(64, ReLU) → Dropout(0.3) → Sigmoid                                                            | max_words=20000, max_len=250, batch=256, epochs=10, recurrent_dropout=0 (CuDNN) |
| BiLSTM | Embedding(20000, 128) → SpatialDropout1D(0.2) → Bidirectional(LSTM(64, dropout=0.2, recurrent_dropout=0)) → Dense(64, ReLU) → Dropout(0.3) → Sigmoid                                             | max_words=20000, max_len=250, batch=256, epochs=10, recurrent_dropout=0 (CuDNN) |
| BERT   | bert-base-uncased → [CLS] pooler → Linear(768, 2); AdamW with linear warmup scheduling; mixed-precision (AMP) training; gradient clipping; on-the-fly tokenization; full dataset (~242K samples) | max_len=256, batch=8, epochs=3, lr=2e-5, warmup=10%, weight_decay=0.01          |

### 4.4. Evaluation Framework Design

A standardized evaluation framework is applied consistently across all models and both datasets, generating:

- Full classification report with per-class Precision, Recall, F1-Score
- Confusion matrix (raw counts and/or normalized)
- ROC curve with AUC score (where applicable)
- Precision-Recall curve with Average Precision score (where applicable)
- Training/validation accuracy and loss curves (for deep learning models)

## 5. Implementation

### 5.1. Depression Dataset Pipeline

#### 5.1.1. Data Preprocessing

The preprocessing module serves as the foundation for all subsequent model training on the Depression dataset. It implements a light preprocessing strategy specifically tailored for social media text (tweets), retaining sentiment-bearing punctuation while removing noise that impedes learning. The module produces eight output artefacts consumed by all downstream model scripts.

Key implementation details:

- **Dataset:** Sentiment140 CSV
- **Preprocessing:** `light_preprocess()` function applies URL/mention tokenization, character deduplication, and punctuation spacing
- **Train/test split:** 90/10 stratified, `random_state=42`
- **Tokenizer:** Keras Tokenizer (`max_nb_words=200,000`, `filters=""`) preserving punctuation tokens
- **Sequences:** Post-padded to `max_sequence_length=50` tokens
- **TF-IDF:** `ngram_range=(1,2)`, `max_features=100,000`, `min_df=3`
- **Outputs:** `x_train_sequences.npy`, `x_test_sequences.npy`, `y_train.npy`, `y_test.npy`, `X_train_tfidf.npz`, `X_test_tfidf.npz`, `tokenizer.pickle`, `label_encoder.pickle`, `tfidf_vectorizer.pickle`

The centralized preprocessing design ensures that all Depression models consume identical data representations, eliminating preprocessing variation as a confounding factor in performance comparisons.

#### 5.1.2. Classical ML Models

##### Logistic Regression

The Logistic Regression model was implemented as a fully custom classifier built from scratch, rather than relying on a pre-built library implementation. This design decision was driven by the need for memory-efficient training on the large-scale TF-IDF matrix (~1.44 million samples × 100,000 features), which would be infeasible to process entirely in memory as a dense array.

The model begins by loading the pre-generated TF-IDF sparse matrices and label arrays produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split used by all other models in the Depression pipeline.

The classifier accepts four configurable hyperparameters: learning rate, number of training iterations, mini-batch size, and a verbosity flag. Internally, the model maintains a weight vector, a scalar bias term, and a history of training losses for convergence monitoring. The sigmoid activation function incorporates numerical clipping to prevent overflow in the exponential computation, ensuring stability on extreme feature values. The loss function implements standard binary cross-entropy with epsilon clipping to avoid logarithm-of-zero errors.

Training is performed using mini-batch stochastic gradient descent, processing the sparse TF-IDF matrix in configurable batches of 5,000 samples per iteration. At the start of each epoch, training indices are randomly permuted for stochastic sampling. For each mini-batch, the forward pass computes the linear model via a sparse matrix–vector dot product followed by sigmoid activation and binary cross-entropy loss computation. The backward pass computes the weight gradient using a sparse transpose operation — a critical optimization that avoids converting the TF-IDF matrix to dense format, keeping memory usage proportional to the number of non-zero entries rather than the full matrix dimensions. Weight and bias parameters are updated using the standard gradient descent update rule. Garbage collection is invoked periodically during training to release accumulated intermediate memory. The final model uses a learning rate of 0.5, a batch size of 5,000, and 1,000 training epochs.

Both probability estimation and class prediction operate in batches to prevent memory overflow when scoring the full dataset. Probability scores are produced via the sigmoid function, which are then thresholded at 0.5 to produce binary class labels.

The evaluation pipeline covers both training and test sets using a comprehensive suite of metrics: Accuracy, Precision, Recall, and F1-Score, along with a detailed per-class classification report. The confusion matrix is visualized as a pair of heatmaps displaying both raw counts and row-normalized percentages, with Specificity and Sensitivity explicitly computed. The Receiver Operating Characteristic curve is plotted with its corresponding AUC score alongside a random-classifier baseline. A Precision-Recall curve with Average Precision score is also generated with a class-balance baseline. A summary bar

chart compares training versus test performance across all four-core metrics. The trained model is serialized for downstream use or dashboard reporting.

## **Naïve Bayes**

The Naïve Bayes model was implemented as a memory-efficient Multinomial Naïve Bayes classifier with Laplace smoothing, designed to operate entirely on sparse matrix representations without converting to dense format. This design ensures scalability on the large-scale TF-IDF matrix ( $\sim 1.44$  million samples  $\times$  100,000 features) within constrained RAM environments.

The model begins by loading the pre-generated TF-IDF sparse matrices and label arrays produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split used by all other models in the Depression pipeline.

The classifier accepts a single configurable hyperparameter: the smoothing parameter (alpha). Internally, the model maintains arrays for class log-prior probabilities and class-conditional feature log-probabilities. Training proceeds by iterating over each class independently: for each class, a Boolean mask isolates the corresponding subset of training samples, and feature counts are accumulated directly from the sparse matrix using a column-wise sum operation — a critical design choice that avoids any dense conversion. Laplace smoothing is applied by adding the alpha constant to all feature counts, and the smoothed counts are normalized and converted to log-probabilities. Class prior probabilities are computed as the proportion of samples belonging to each class and stored in log space. Memory is explicitly freed after processing each class through garbage collection. The final model uses a smoothing parameter of 0.5.

Prediction is performed entirely in batches to prevent memory overflow when scoring the full dataset. Log-probability estimation computes the joint log-likelihood for each sample by multiplying the sparse input matrix against the transposed feature log-probability matrix and adding the class log-prior — effectively computing the log-posterior for each class in a single sparse matrix operation. Class labels are assigned by selecting the class with the highest log-posterior. For probability calibration, raw log-probabilities are converted to proper probability distributions using the log-sum-exp trick, which subtracts the per-sample maximum log-probability before exponentiation to prevent numerical overflow, then normalizes to ensure probabilities sum to one.

The evaluation pipeline covers both training and test sets using a comprehensive suite of metrics: Accuracy, Precision, Recall, and F1-Score, along with a detailed per-class classification report. The confusion matrix is visualized as a pair of heatmaps displaying both raw counts and row-normalized percentages, with Specificity and Sensitivity explicitly computed. The Receiver Operating Characteristic curve is plotted with its corresponding AUC score alongside a random-classifier baseline. A Precision-Recall curve with Average Precision score is also generated with a class-balance baseline. Both the ROC and Precision-Recall curves are computed on stratified 50,000-sample subsets to maintain memory efficiency on the large dataset. A summary bar chart compares training versus test performance across all four-core metrics.

## **SVM**

The Support Vector Machine model was implemented using a linear support vector classifier trained with the primal formulation, optimized for high-dimensional sparse TF-IDF features. The primal formulation (`dual=False`) was selected over the dual formulation as it is more efficient when the number of features exceeds the number of samples, which is the case with the 100,000-feature TF-IDF representation.

The model begins by loading the pre-generated TF-IDF sparse matrices and label arrays produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split used by all other models in the Depression pipeline.

The optimal hyperparameters — a regularization parameter of 0.1 and balanced class weighting — were identified through a prior grid search with cross-validation. The balanced class weighting automatically adjusts class weights inversely proportional to class frequencies, mitigating potential bias towards the majority class. The maximum iteration limit is set to 3,000 to ensure convergence on the large-scale dataset with a fixed random seed of 42 for reproducibility.

Since the linear support vector classifier does not natively output probability estimates, probability calibration is applied post-training using Platt scaling (sigmoid method). The calibration is performed on a 100,000-sample subset of the training data to maintain memory efficiency, fitting a sigmoid function to the decision function outputs of the already-trained model to produce well-calibrated probability scores.

Predictions and probability estimates are computed on both training and test sets, with garbage collection invoked between sets to manage memory. The calibrated probability scores for the positive class are extracted for use in threshold-dependent evaluation metrics.

The evaluation pipeline covers both training and test sets using a comprehensive suite of metrics: Accuracy, Precision, Recall, and F1-Score, along with a detailed per-class classification report. The confusion matrix is visualized as a pair of heatmaps displaying both raw counts and row-normalized percentages, with Specificity and Sensitivity explicitly computed. The Receiver Operating Characteristic curve is plotted with its corresponding AUC score alongside a random-classifier baseline. A Precision-Recall curve with Average Precision score is also generated with a class-balance baseline. A summary bar chart compares training versus test performance across all four-core metrics.

## **Random Forest**

The Random Forest model was implemented as an ensemble of decision trees with a custom early stopping mechanism based on Out-of-Bag (OOB) error monitoring, designed to automatically determine the optimal number of trees without requiring a separate validation set.

The model begins by loading the pre-generated TF-IDF sparse matrices and label arrays produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split used by all other models in the Depression pipeline.

Due to the computational expense of training a Random Forest on the full 1.44 million-sample dataset, the training set is sub-sampled to 200,000 examples using stratified sampling. The stratification is performed manually: equal numbers of positive and negative samples (100,000 each) are drawn independently from each class and then concatenated and shuffled, preserving the original 50/50 class balance. A fixed random seed of 42 ensures reproducibility of the sampling.

The early stopping mechanism leverages warm starting — the ability to incrementally grow the forest by adding new trees to the existing ensemble without retraining from scratch. The forest begins with an initial ensemble of 50 trees (the minimum for reliable OOB estimates) and grows by 10 trees per round, up to a hard cap of 300 trees. After each round, the OOB error — computed from samples not included in each tree's bootstrap sample — is recorded. If the OOB error fails to

improve by at least 0.0001 for 3 consecutive rounds, training is stopped early. This approach provides a data-driven method for selecting the ensemble size, avoiding both underfitting (too few trees) and unnecessary computation (too many trees with diminishing returns). Once the optimal number of estimators is identified, a clean final model is re-fitted from scratch with that exact tree count to ensure a consistent, production-ready model state.

Each tree in the ensemble uses the square root of the total feature count as the number of candidate features at each split, a standard heuristic for classification that promotes diversity among trees. Trees are allowed to grow to full depth with no maximum depth constraint, while a minimum leaf size of 5 samples provides slight regularization to reduce overfitting. Class weighting is omitted because the sub-sampled training set is already perfectly balanced. All available CPU cores are utilized via parallel training to maximize throughput.

Feature importances are extracted from the final trained model using Gini importance — the total decrease in node impurity contributed by each feature across all trees — and the top 20 most important features are visualized using a horizontal bar chart. This provides model interpretability insights into which TF-IDF features (words and bigrams) are most discriminative for sentiment classification.

Predictions and probability estimates are computed on both the sub-sampled training set and the full test set. Probability scores are produced via the proportion of trees voting for each class.

The evaluation pipeline covers both training and test sets using a comprehensive suite of metrics: Accuracy, Precision, Recall, and F1-Score, along with a detailed per-class classification report. The confusion matrix is visualized as a pair of heatmaps displaying both raw counts and row-normalized percentages, with Specificity and Sensitivity explicitly computed. The Receiver Operating Characteristic curve is plotted with its corresponding AUC score alongside a random-classifier baseline. A Precision-Recall curve with Average Precision score is also generated with a class-balance baseline. A summary bar chart compares training versus test performance across all four-core metrics.

### **5.1.3. Deep Learning Models**

#### **GloVe Embedding Loading**

All three deep learning models use an identical `load_glove_embeddings()` function. The function reads the `glove.twitter.27B.100d.txt` file line-by-line, constructing a word-to-vector mapping. An embedding matrix of shape `(vocab_size, 100)` is populated for all vocabulary words present in the GloVe index. GloVe coverage — the percentage of vocabulary words found in the GloVe index — is reported at each run, providing a quality indicator for the embedding initialization.

#### **CNN**

The Convolutional Neural Network model was implemented using a hybrid embedding architecture built with the Keras Functional API, designed to capture localized n-gram patterns in tweet text through convolutional filters while combining pre-trained semantic knowledge with domain-adaptive representation learning.

The model begins by loading the pre-generated integer sequence arrays, label arrays, tokenizer, and label encoder produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split and vocabulary used by all other deep learning models in the Depression pipeline.

The embedding layer employs a dual-branch hybrid strategy. The first branch loads the GloVe Twitter 27B 100-dimensional pre-trained word vectors into a frozen (non-trainable) embedding layer, providing a strong semantic foundation grounded in general-purpose Twitter language. The embedding matrix is constructed by iterating over the vocabulary and populating a matrix with the corresponding GloVe vector for each word found in the pre-trained index; words not found in GloVe are initialized as zero vectors. The GloVe coverage — the percentage of vocabulary words with a matching pre-trained vector — is reported as a quality indicator. The second branch is a fully trainable 200-dimensional embedding layer that learns domain-specific representations during training, allowing the model to capture sentiment-specific patterns not present in the general-purpose GloVe vectors. The outputs of both branches are concatenated along the feature axis, producing a merged 300-dimensional embedding for each token.

After concatenation, dropout at a rate of 0.4 is applied for regularization. The core feature extraction is performed by a single one-dimensional convolutional layer with 128 filters and a kernel size of 5, using ReLU activation. This configuration captures local 5-gram patterns — contiguous sequences of five tokens — enabling the model to detect discriminative phrases and word combinations associated with sentiment. Global max pooling then aggregates the most salient feature across the entire sequence for each filter, reducing the variable-length feature maps to a fixed-length 128-dimensional vector. A fully connected layer with 64 units and ReLU activation provides further abstraction, followed by a second dropout layer at 0.4. The final output layer uses softmax activation for multi-class classification.

The model is compiled with the Adam optimizer at a learning rate of 0.001 and sparse categorical cross-entropy loss. Training runs for a maximum of 15 epochs with a batch size of 512, optimized for the T4 GPU memory profile. A 5% validation split is held out from the training data for monitoring. Three callbacks regulate training: early stopping with patience of 2 epochs restores the best weights when validation loss ceases to improve, learning rate reduction on plateau decreases the learning rate by a factor of 0.2 after 2 epochs without improvement (with a minimum of 0.00001), and model checkpointing saves the best model based on validation loss. The trained model is serialized for downstream use.

## **LSTM**

The Long Short-Term Memory model was implemented as a high-capacity recurrent architecture built with the Keras Functional API, designed to capture long-range sequential dependencies in tweet text through gated memory cells while employing the same hybrid embedding strategy used across all deep learning models in the Depression pipeline.

The model begins by loading the pre-generated integer sequence arrays, label arrays, tokenizer, and label encoder produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split and vocabulary used by all other deep learning models in the Depression pipeline.

The embedding layer employs the identical dual-branch hybrid strategy used by the CNN model. The first branch loads the GloVe Twitter 27B 100-dimensional pre-trained word vectors into a frozen (non-trainable) embedding layer, providing general-purpose semantic grounding from

Twitter language. The embedding matrix is constructed by iterating over the vocabulary and populating each entry with the corresponding GloVe vector where available; words without a match are initialized as zero vectors, and the GloVe coverage percentage is reported. The second branch is a fully trainable 200-dimensional embedding layer that learns domain-specific sentiment representations during training. The outputs of both branches are concatenated, producing a merged 300-dimensional embedding for each token.

After concatenation, dropout at a rate of 0.4 is applied for regularization. The core sequential processing is performed by a single LSTM layer with 256 units — a high-capacity configuration chosen to capture nuanced temporal patterns from the large vocabulary. The LSTM layer applies recurrent dropout at a rate of 0.1 to regularize the recurrent connections, reducing overfitting on the sequential state transitions. The layer returns only the final hidden state (`return_sequences=False`), producing a single 256-dimensional vector that encodes the entire input sequence. A fully connected layer with 64 units and ReLU activation provides further abstraction, followed by a second dropout layer at 0.4. The final output layer uses softmax activation for multi-class classification.

The model is compiled with the Adam optimizer at a learning rate of 0.001 and sparse categorical cross-entropy loss. Training runs for a maximum of 15 epochs with a batch size of 512, optimized for the T4 GPU memory profile. A 5% validation split is held out from the training data for monitoring. Three callbacks regulate training: early stopping with patience of 2 epochs restores the best weights when validation loss ceases to improve, learning rate reduction on plateau decreases the learning rate by a factor of 0.2 after 2 epochs without improvement (with a minimum of 0.00001), and model checkpointing saves the best model based on validation loss. The trained model is serialized for downstream use.

## **BiLSTM**

The Bidirectional LSTM model was implemented as a high-capacity recurrent architecture built with the Keras Functional API, extending the standard LSTM by processing each input sequence in both forward and backward directions. This bidirectional context modelling is particularly well-suited for short social media text where the beginning and end of a post are equally informative for sentiment classification.

The model begins by loading the pre-generated integer sequence arrays, label arrays, tokenizer, and label encoder produced by the centralized preprocessing pipeline, ensuring it operates on the exact same data split and vocabulary used by all other deep learning models in the Depression pipeline.

The embedding layer employs the identical dual-branch hybrid strategy used by the CNN and LSTM models. The first branch loads the GloVe Twitter 27B 100-dimensional pre-trained word vectors into a frozen (non-trainable) embedding layer, providing general-purpose semantic grounding from Twitter language. The embedding matrix is constructed by iterating over the vocabulary and populating each entry with the corresponding GloVe vector where available; words without a match are initialized as zero vectors, and the GloVe coverage percentage is reported. The second branch is a fully trainable 200-dimensional embedding layer that learns domain-specific sentiment representations during training. The outputs of both branches are concatenated, producing a merged 300-dimensional embedding for each token.

After concatenation, dropout at a rate of 0.4 is applied for regularization. The core sequential processing is performed by wrapping a 256-unit LSTM layer inside a Bidirectional wrapper. This processes each tweet in both forward and backward directions simultaneously, effectively doubling the output dimensionality to 512. The forward pass captures how sentiment builds from the start of the text, while the backward pass captures context from the end, and the concatenated outputs provide a richer representation than unidirectional processing alone. The inner LSTM applies recurrent dropout at a rate of 0.1 to regularize the recurrent connections. The layer returns only the final concatenated hidden states (`return_sequences=False`), producing a single 512-dimensional vector encoding the full bidirectional context. A fully connected layer with 64 units and ReLU activation provides further abstraction, followed by a second dropout layer at 0.4. The final output layer uses softmax activation for multi-class classification.

The model is compiled with the Adam optimizer at a learning rate of 0.001 and sparse categorical cross-entropy loss. Training runs for a maximum of 15 epochs with a batch size of 512, optimized for the T4 GPU memory profile. A 5% validation split is held out from the training data for monitoring. Three callbacks regulate training: early stopping with patience of 2 epochs restores the best weights when validation loss ceases to improve, learning rate reduction on plateau decreases the learning rate by a factor of 0.2 after 2 epochs without improvement (with

a minimum of 0.00001), and model checkpointing saves the best model based on validation loss. The trained model is serialized for downstream use.

#### **5.1.4. BERT**

The BERT model was implemented using a PyTorch-based fine-tuning pipeline built on the Hugging Face Transformers library, leveraging the pre-trained bert-base-uncased model as the foundation for binary sentiment classification. Unlike the other deep learning models in the Depression pipeline which use TensorFlow/Keras, BERT requires a PyTorch training loop to provide fine-grained control over mixed-precision training, gradient clipping, and learning rate scheduling.

The model begins by loading the raw Sentiment140 CSV dataset directly, rather than consuming the centralized preprocessing artefacts, because BERT requires its own WordPiece tokenisation rather than the integer sequences used by CNN, LSTM, and BiLSTM. A dedicated text preprocessing function normalizes each tweet by HTML-unescaping, lowercasing, replacing URLs and user mentions with special tokens [URL] and [USER] respectively, and expanding common English contractions (e.g., "won't" → "will not", "I'm" → "I am") using a dictionary of 48 contraction mappings applied via a compiled regular expression. The preprocessed dataset is split into training (85%), validation (5%), and test (10%) subsets using stratified splitting with a fixed random seed of 42, and cached to disk as pickle files for efficient reloading.

Tokenization is handled by the fast BERT tokenizer, with the vocabulary extended to include the two custom special tokens [URL] and [USER]. A custom dataset class wraps each tweet into BERT-compatible input tensors: input IDs and attention masks are produced by the tokenizer with a maximum sequence length of 48, using max-length padding and truncation. Three data loaders are constructed for training (shuffled), validation, and test sets with a batch size of 128 and pinned memory for GPU transfer efficiency.

The model architecture consists of the pre-trained BERT encoder followed by a dropout layer (rate 0.1) and a single linear classification head mapping from BERT's 768-dimensional hidden size to 2 output classes. The model's token embedding layer is resized to accommodate the two added special tokens. Classification is performed on the [CLS] token's pooler output — a single 768-dimensional vector that encodes the entire input sequence through BERT's self-attention mechanism.

Training employs the AdamW optimizer with a learning rate of  $2e-5$  and weight decay of 0.01. A linear warmup schedule warms the learning rate over the first 6% of total training steps before linearly decaying to zero. The loss function is cross-entropy with label smoothing at 0.1 to improve generalization by preventing overconfident predictions. Mixed-precision training via automatic mixed precision (AMP) and gradient scaling is enabled to reduce GPU memory consumption and accelerate computation on the T4 GPU. Gradient norms are clipped to 1.0 to prevent exploding gradients during fine-tuning. A custom early stopping mechanism monitors validation loss with patience of 2 epochs, saving the best model checkpoint to disk and restoring it upon completion. Training runs for a maximum of 5 epochs. Full reproducibility is ensured by seeding Python's random module, NumPy, PyTorch (CPU and CUDA), and setting cuDNN to deterministic mode.

The evaluation pipeline performs a full inference pass over the held-out test set with gradient computation disabled. Predictions are produced by taking the argmax of the raw logits, and probability scores for the positive class are obtained via softmax for threshold-dependent metrics. The classification report provides per-class Precision, Recall, and F1-Score alongside overall accuracy. The confusion matrix is visualized as a normalized heatmap. The ROC curve is plotted with its corresponding AUC score. Training history (loss and accuracy for both training and validation sets) is plotted across epochs. A comprehensive dashboard JSON file is exported containing all evaluation metrics, confusion matrix values, sampled ROC curve points, training history, and dataset statistics for downstream dashboard visualization.

## **5.2. Suicide Dataset Pipeline**

Unlike the Depression pipeline which uses a centralized preprocessing script to generate shared artefacts, the Suicide models follow a self-contained pattern where each model script independently handles its own data loading, merging, preprocessing, training, and evaluation. This design accommodates the Google Colab execution environment where each notebook runs as an independent session.

### **5.2.1. Data Loading, Merging and Preprocessing**

All Suicide model scripts implement an identical data ingestion pattern. Two publicly available datasets are loaded from Google Drive: the primary Suicide Detection dataset (~232,000 entries) and a supplementary suicide-only dataset (~10,000 entries) extracted from the Reddit Depression and SuicideWatch corpus. The primary dataset's

class column is renamed to a standardized label column and mapped from string values to binary integers ("suicide" → 1, "non-suicide" → 0). The supplementary dataset retains only the text column and is assigned a label of 1. Both datasets are concatenated into a single combined dataframe and rows with missing values are dropped, producing a merged dataset of approximately 242,066 samples (126,029 suicide, 116,037 non-suicide).

Text preprocessing applies a shared lightweight cleaning function across all models except Random Forest and BERT. Text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, all punctuation is removed using a character translation table, and excess whitespace is collapsed to single spaces. The BERT model applies the same URL, mention, and hashtag removal steps but omits punctuation stripping, since BERT's WordPiece tokenizer processes punctuation as meaningful subword tokens. The Random Forest model bypasses text cleaning entirely and instead extracts hand-crafted features directly from the raw text.

For classical ML models (Naïve Bayes, SVM, Logistic Regression), the cleaned text is vectorized using TF-IDF with a maximum vocabulary of 20,000 features. For deep learning models (CNN, LSTM, BiLSTM), tokenization is performed using a Keras Tokenizer with a vocabulary of 20,000 words, and integer sequences are post-padded to a uniform length of 250 tokens. For BERT, tokenization is handled by the bert-base-uncased WordPiece tokenizer with a maximum sequence length of 256 tokens. All models split the data into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42, preserving the original class distribution.

### **5.2.2. Classical ML Models**

#### **Logistic Regression**

The Logistic Regression model for the Suicide pipeline was implemented as a custom binary classifier built from scratch using gradient descent with L2 regularization, following the self-contained pattern where data loading, merging, preprocessing, TF-IDF feature extraction, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The supplementary dataset is prepared by filtering for suicide-

labelled entries from the source corpus and applying a lightweight cleaning function that lowercases the text, removes URLs, user mentions, and non-alphanumeric characters via regular expression matching, and collapses excess whitespace. The primary dataset's class labels are mapped from string values to binary integers ("suicide" → 1, "non-suicide" → 0), and the supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe. Duplicate entries and empty rows are removed to ensure data quality. Text preprocessing on the primary dataset applies lowercasing, URL removal, stripping of mention and hashtag symbols, and punctuation removal using a character translation table. The combined data is split into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42. Feature extraction is performed using TF-IDF vectorization with a maximum vocabulary of 20,000 features, unigram and bigram n-gram range, and English stop word removal. The resulting sparse matrices and label arrays are serialized for reproducible reloading.

The classifier is implemented entirely from scratch without relying on a pre-built library implementation. The model accepts five configurable hyperparameters: learning rate, number of training iterations, regularization type (L1 or L2), regularization strength (lambda), and an optional batch size parameter. Weights are initialized to zero and the bias term is initialized to zero. The sigmoid activation function includes numerical clipping to prevent overflow in the exponential computation.

Training is performed using full-batch gradient descent on the sparse TF-IDF matrix. At each iteration, the predicted probabilities are computed via the sigmoid of the linear decision function. The weight gradient is calculated as the mean of the product of the error vector and the transposed feature matrix, with the gradient correctly handled for sparse matrix outputs. L2 regularization adds a penalty proportional to the weight magnitudes scaled by lambda divided by the number of samples. Weights and bias are updated using the standard gradient descent rule. The binary cross-entropy loss with the regularization penalty is computed and logged every 100 iterations for convergence monitoring. The final model uses a learning rate of 0.5 — selected for faster convergence within the Colab session constraints — 1,000 iterations, L2 regularization with a lambda of 0.01, and a default decision threshold of 0.5.

The evaluation pipeline reports Accuracy, Precision, Recall, and F1-Score computed from a manually constructed confusion matrix. The confusion matrix is visualized as a heatmap displaying raw counts. A training loss convergence curve is plotted across iterations, and threshold tuning is performed by evaluating F1-Score at five candidate thresholds (0.3, 0.4, 0.5, 0.6, 0.7) to identify the optimal decision boundary.

## **Naïve Bayes**

The Naïve Bayes model for the Suicide pipeline was implemented as a Multinomial Naïve Bayes classifier built from scratch, following the self-contained pattern where data loading, merging, preprocessing, feature extraction, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values ("suicide" and "non-suicide") to binary integers (1 and 0 respectively). The supplementary dataset contains only suicide-related text and is assigned a label of 1. Both datasets are standardized to a common schema consisting of text and label columns, concatenated into a single combined dataframe, and any rows with missing values are dropped.

Text preprocessing applies a lightweight cleaning function: text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, all punctuation is removed using a character translation table, and excess whitespace is collapsed to single spaces. The cleaned text is then split into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42 to preserve the original class distribution. Feature extraction is performed using TF-IDF vectorization with a maximum vocabulary of 20,000 features, producing sparse term-frequency inverse-document-frequency matrices for both training and test sets.

The classifier is implemented from scratch without relying on a pre-built library implementation. The model accepts a single hyperparameter: the Laplace smoothing constant (alpha), set to the default value of 1.0. During training, the model iterates over each class independently. For each class, the prior probability is computed as the proportion of training samples belonging to that class and stored in log space. Class-conditional feature probabilities are calculated by summing feature

values across all samples of that class from the sparse TF-IDF matrix, applying Laplace smoothing by adding alpha to each feature count and normalizing by the total count plus alpha times the number of features, then converting to log-probabilities. This denominator-level smoothing prevents zero-probability estimates for features not observed in a particular class.

Prediction computes the joint log-likelihood for each test sample by multiplying the sparse TF-IDF matrix against the transposed feature log-probability matrix and adding the class log-priors. Class labels are assigned by selecting the class with the highest joint log-likelihood via `argmax`.

The evaluation pipeline generates a detailed classification report with per-class Precision, Recall, and F1-Score for both the Non-Suicide and Suicide classes. The confusion matrix is visualized as a heatmap displaying raw counts.

## **SVM**

The Support Vector Machine model for the Suicide pipeline was implemented as a custom Linear SVM classifier built from scratch using hinge loss and sub gradient descent, following the self-contained pattern where data loading, merging, preprocessing, class balancing, training, and evaluation are all encapsulated within a single script.

The model begins by loading pre-generated TF-IDF sparse matrices and label arrays for the combined Suicide dataset. String labels are mapped to binary integers ("suicide"  $\rightarrow$  1, "non-suicide"  $\rightarrow$  0). An analysis of the class distribution reveals an imbalance between suicide and non-suicide samples, which is addressed through up sampling: the minority class is resampled with replacement to match the majority class count, producing a perfectly balanced training set. The balanced indices are shuffled with a fixed random seed of 42 for reproducibility.

The classifier is implemented entirely from scratch without relying on a pre-built library implementation. The model accepts four configurable hyperparameters: learning rate, L2 regularization parameter ( $\lambda$ ), number of training iterations, and mini-batch size. Weights are initialized with small random values drawn from a standard normal distribution scaled by 0.01, and the bias term is initialized to zero. The model detects whether the input is a sparse matrix and adapts its matrix operations accordingly, using sparse dot products when available.

Training is performed using mini-batch sub gradient descent. Labels are internally converted from  $\{0, 1\}$  to  $\{-1, +1\}$  for the hinge loss formulation. At each iteration, a random mini-batch of 10,000 samples is drawn without replacement. For each mini-batch, the decision scores are computed via a sparse matrix–vector dot product plus the bias term. The hinge loss sub gradient is computed by identifying support vectors — samples whose margin (the product of the label and score) falls below 1. For misclassified or margin-violating samples, the weight gradient combines L2 regularization with the average of the misclassified sample contributions computed via a sparse transpose operation. For correctly classified samples outside the margin, only the L2 regularization gradient is applied. Weights and bias are updated using the standard gradient descent rule. Training accuracy, the number of support vectors, and prediction distribution are logged every 100 iterations for convergence monitoring. The final model uses a learning rate of 0.5, a regularization parameter of 0.00001, a batch size of 10,000, and 1,000 training iterations.

Prediction computes the decision scores via sparse matrix–vector multiplication plus bias, and class labels are assigned by thresholding at zero: non-negative scores are mapped to class 1 (suicide) and negative scores to class 0 (non-suicide).

The evaluation pipeline reports Accuracy, Precision, Recall, and F1-Score. The confusion matrix is visualized as a heatmap displaying raw counts. A PCA reduction to two dimensions is applied to a 5,000-sample subset of test predictions for visual inspection of the decision boundary separation.

## **Random Forest**

The Random Forest model for the Suicide pipeline was implemented entirely from scratch — including the underlying decision tree and ensemble logic — without relying on any pre-built library implementation. The model follows the self-contained pattern where data loading, merging, feature extraction, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values to binary integers ("suicide"  $\rightarrow$  1, "non-suicide"  $\rightarrow$  0). The supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe.

Unlike the other Suicide models which use TF-IDF vectorization, the Random Forest model employs a custom hand-crafted feature extraction pipeline that generates 13 engineered features from each text sample. These features span three categories: structural features (character count, word count, average word length, sentence count, punctuation count, digit count, capital letter ratio), sentiment features (TextBlob polarity and subjectivity scores, VADER negative/positive/neutral compound scores), and a domain-specific suicide keyword count based on a curated list of 13 crisis-related terms. The data is split into training (80%) and test (20%) subsets using a custom random split function.

The decision tree implementation uses Gini impurity as the splitting criterion. At each node, a random subset of features (square root of total features) is selected, and the best split is identified by evaluating candidate thresholds at the 10th through 90th percentiles of feature values — an optimization that reduces computation compared to evaluating all unique values. Splitting terminates when the maximum depth is reached, only one class remains, or the node contains fewer samples than the minimum size threshold, in which case the majority class is returned as the leaf prediction.

The Random Forest ensemble is constructed by training multiple independent decision trees on bootstrap samples — random samples drawn with replacement from the training set, each the same size as the original. Each tree is built with the random feature subset selection at every split, ensuring diversity among trees. Final predictions are produced via majority voting across all trees, with the class receiving the most votes assigned as the predicted label.

The final model uses 15 trees, a maximum depth of 6, and a minimum leaf size of 20. The evaluation pipeline reports Accuracy, Precision, Recall, and F1-Score. The confusion matrix is visualized as a heatmap displaying raw counts, alongside a prediction distribution bar chart comparing actual versus predicted class counts.

### **5.2.3. Deep Learning Models**

#### **CNN**

The Convolutional Neural Network model for the Suicide pipeline was implemented using a sequential Keras architecture with a trainable embedding layer, following the self-contained pattern where data loading, merging, preprocessing, tokenization, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values to binary integers ("suicide"  $\rightarrow$  1, "non-suicide"  $\rightarrow$  0). The supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe, and rows with missing values are dropped.

Text preprocessing applies a lightweight cleaning function: text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, all punctuation is removed using a character translation table, and excess whitespace is collapsed to single spaces. Tokenization is performed using a Keras Tokenizer with a maximum vocabulary of 20,000 words, and the cleaned text is converted to integer sequences. These sequences are post-padded to a uniform length of 250 tokens, accommodating the longer Reddit posts typical of the Suicide dataset. The data is split into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42.

The model architecture follows a sequential design. The embedding layer maps each of the 20,000 vocabulary tokens into a 128-dimensional trainable vector space, allowing the model to learn task-specific word representations during training. Spatial dropout at a rate of 0.3 is applied to the embedding output, randomly zeroing entire embedding channels to regularize the feature maps and reduce overfitting. A single one-dimensional convolutional layer with 128 filters and a kernel size of 5 using ReLU activation captures local 5-gram patterns in the text. Global max pooling aggregates the most salient feature across the entire sequence for each filter, producing a fixed-length 128-dimensional vector. A fully connected layer with 128 units and ReLU activation provides further abstraction, followed by standard dropout at a rate of 0.5. The output layer uses a single unit with sigmoid activation for binary classification.

The model is compiled with the Adam optimizer and binary cross-entropy loss. Training runs for a maximum of 10 epochs with a batch size of 64. A 10% validation split is held out from the training data for monitoring. Early stopping with patience of 3 epochs monitors validation loss and restores the best weights when improvement ceases. The evaluation pipeline generates a detailed classification report and the confusion matrix is visualized as a heatmap. Training and validation loss curves are plotted across epochs.

## LSTM

The Long Short-Term Memory model for the Suicide pipeline was implemented using a sequential Keras architecture with a trainable embedding layer, following the self-contained pattern where data loading, merging, preprocessing, tokenization, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values to binary integers ("suicide"  $\rightarrow$  1, "non-suicide"  $\rightarrow$  0). The supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe, and rows with missing values are dropped. Text preprocessing applies a lightweight cleaning function: text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, all punctuation is removed using a character translation table, and excess whitespace is collapsed to single spaces. Tokenization is performed using a Keras Tokenizer with a maximum vocabulary of 20,000 words, and the cleaned text is converted to integer sequences. These sequences are post-padded to a uniform length of 250 tokens, accommodating the longer Reddit posts typical of the Suicide dataset. The data is split into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42.

The model architecture follows a sequential design. The embedding layer maps each of the 20,000 vocabulary tokens into a 128-dimensional trainable vector space, allowing the model to learn task-specific word representations during training. Spatial dropout at a rate of 0.2 is applied to the embedding output, randomly zeroing entire embedding channels to regularize the feature maps and reduce overfitting. The core sequential processing is performed by a single LSTM layer with 64 units, which captures long-range temporal dependencies through its gated memory cell mechanism. The LSTM layer applies dropout at a rate of 0.2 to the input connections, while recurrent dropout is set to zero to enable CuDNN GPU acceleration — a deliberate optimization that prioritizes training speed on the T4 GPU over additional recurrent regularization. The LSTM layer returns only the final hidden state, producing a single 64-dimensional vector encoding the entire input sequence. A fully connected layer with 64 units and ReLU activation provides further abstraction, followed by standard dropout at a rate of 0.3. The output layer uses a single unit with sigmoid activation for binary classification.

The model is compiled with the Adam optimizer and binary cross-entropy loss. Training runs for a maximum of 10 epochs with a batch size of 256. A 10% validation split is held out from the training data for monitoring. Early stopping with patience of 3 epochs monitors validation loss and restores the best weights when improvement ceases.

The evaluation pipeline generates predictions by thresholding the sigmoid output probabilities at 0.5. A detailed classification report provides per-class Precision, Recall, and F1-Score for the Non-Suicide and Suicide classes. The confusion matrix is visualized as a heatmap displaying raw counts. Training and validation loss and accuracy curves are plotted across epochs to assess overfitting behavior. Additionally, a prediction error analysis identifies false positive and false negative samples, decoding the padded integer sequences back to text using the reverse word index for qualitative inspection of misclassified examples.

## **BiLSTM**

The Bidirectional LSTM model for the Suicide pipeline was implemented using a sequential Keras architecture, extending the standard LSTM by wrapping it in a Bidirectional layer that processes each input sequence in both forward and backward directions. This bidirectional context modelling is particularly well-suited for the longer Reddit posts in the Suicide dataset, where contextual cues at both the beginning and end of a post are equally informative for classification. The model follows the self-contained pattern where data loading, merging, preprocessing, tokenization, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values to binary integers ("suicide" → 1, "non-suicide" → 0). The supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe, and rows with missing values are dropped. Text preprocessing applies a lightweight cleaning function: text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, all punctuation is removed using a character translation table, and excess whitespace is collapsed to single spaces. Tokenization is performed using a Keras Tokenizer with a maximum vocabulary of 20,000 words, and the cleaned text is converted to integer sequences. These sequences are post-padded to a uniform length of 250 tokens. The data is split into training (80%) and test (20%) subsets using stratified splitting with a

fixed random seed of 42.

The model architecture follows a sequential design. The embedding layer maps each of the 20,000 vocabulary tokens into a 128-dimensional trainable vector space, allowing the model to learn task-specific word representations during training. Spatial dropout at a rate of 0.2 is applied to the embedding output, randomly zeroing entire embedding channels to regularize the feature maps and reduce overfitting. The core sequential processing is performed by a Bidirectional wrapper around an LSTM layer with 64 units, which processes the input in both forward and backward directions and concatenates the outputs, producing an effective output dimensionality of 128. The LSTM layer applies dropout at a rate of 0.2 to the input connections, while recurrent dropout is set to zero to enable CuDNN GPU acceleration. A fully connected layer with 64 units and ReLU activation provides further abstraction, followed by standard dropout at a rate of 0.3. The output layer uses a single unit with sigmoid activation for binary classification.

The model is compiled with the Adam optimizer and binary cross-entropy loss. Training runs for a maximum of 10 epochs with a batch size of 256. A 10% validation split is held out from the training data for monitoring. Early stopping with patience of 3 epochs monitors validation loss and restores the best weights when improvement ceases.

The evaluation pipeline generates predictions by thresholding the sigmoid output probabilities at 0.5. A detailed classification report provides per-class Precision, Recall, and F1-Score for the Non-Suicide and Suicide classes. The confusion matrix is visualized as a heatmap displaying raw counts.

#### **5.2.4. BERT**

The BERT model for the Suicide pipeline was implemented using a PyTorch-based fine-tuning pipeline built on the Hugging Face Transformers library, leveraging the pre-trained bert-base-uncased model as the foundation for binary suicide/non-suicide classification. The model follows the self-contained pattern where data loading, merging, preprocessing, tokenization, training, and evaluation are all encapsulated within a single script.

The model begins by loading two publicly available datasets: the primary Suicide Detection dataset and a supplementary suicide-only dataset. The primary dataset's class labels are mapped from string values to binary integers ("suicide" → 1, "non-suicide" → 0). The

supplementary dataset is assigned a label of 1. Both datasets are merged into a single combined dataframe, and rows with missing text or label values are dropped. Unlike the previous iteration which subsampled 100,000 entries, the updated implementation uses the full combined dataset (~242,000 entries), shuffled with a fixed random seed of 42 to ensure reproducibility. Text preprocessing applies a lightweight cleaning function: text is lowercased, URLs and web links are removed via regular expression matching, user mentions and hashtag symbols are stripped, and excess whitespace is collapsed to single spaces. The preprocessed dataset is split into training (80%) and test (20%) subsets using stratified splitting with a fixed random seed of 42.

Tokenization is performed on-the-fly during training rather than pre-tokenizing the entire dataset up front. A custom dataset class stores the raw cleaned text and labels, and tokenizes each sample at access time using the BERT tokenizer with a maximum sequence length of 256 tokens, max-length padding, and truncation. This on-the-fly approach significantly reduces memory consumption by avoiding the need to hold all tokenized tensors in RAM simultaneously — a critical optimization when training on the full dataset within Colab's memory constraints. Two data loaders are constructed: a training loader with shuffling enabled and a test loader without shuffling, both using a batch size of 8 — reduced from the Depression pipeline's batch size of 128 to accommodate the longer sequence length (256 vs. 48) and full dataset size within GPU memory constraints.

The model architecture consists of the pre-trained BERT encoder followed by a linear classification head mapping from BERT's 768-dimensional hidden size to 2 output classes, loaded directly via the Hugging Face auto-model interface for sequence classification. Training employs the AdamW optimizer with a learning rate of  $2e-5$  and weight decay of 0.01. A linear warmup schedule warms the learning rate over the first 10% of total training steps before linearly decaying to zero. Mixed-precision training via automatic mixed precision (AMP) and gradient scaling is enabled to reduce GPU memory consumption and accelerate computation on the T4 GPU. Gradient norms are clipped to 1.0 to prevent exploding gradients during fine-tuning. Training runs for 3 epochs. The trained model weights are saved to disk for downstream use.

The evaluation pipeline performs a full inference pass over the held-out test set with gradient computation disabled. Predictions are produced by taking the argmax of the raw logits. The classification report provides

per-class Precision, Recall, and F1-Score for the Non-Suicide and Suicide classes alongside overall accuracy. The confusion matrix is visualized as a heatmap displaying raw counts. A Precision-Recall curve with Average Precision (PR-AUC) score is also generated and saved, computed from the softmax probability scores of the positive class. The ROC curve is plotted with its corresponding AUC score alongside a random-classifier baseline.

### **5.3. Colab-Specific Optimizations**

The following optimizations are applied across both pipelines to ensure reliable execution within Google Colab constraints:

- `BATCH_SIZE=512` selected empirically to maximize T4 GPU utilization without OOM errors (Depression DL models)
- `tf.config.experimental.set_memory_growth` completely removed — Colab T4 manages GPU memory automatically
- Model checkpointing to `.h5` format with `save_best_only=True` preserves best validation epoch
- Garbage collection (`gc.collect()`) called after each major operation in classical ML scripts
- Sparse matrix operations (`load_npz`, dot products) avoid unnecessary dense conversions

## 6. Testing and Evaluation

### 6.1. Evaluation Methodology

All models are evaluated on identical held-out test sets created using stratified train/test splitting with `random_state=42`:

- Depression dataset: 10% held-out test set (~160,000 tweets)
- Suicide dataset: 20% held-out test set (~48,414 posts)

Binary classification metrics are reported for both training and test sets to assess generalization. The following metrics are computed for all models:

- Accuracy: Overall proportion of correct predictions
- Precision: Proportion of true positives among positive predictions
- Recall (Sensitivity): Proportion of actual positives correctly identified
- F1-Score: Harmonic mean of Precision and Recall
- ROC-AUC: Area Under the Receiver Operating Characteristic Curve
- Average Precision (PR-AUC): Area Under the Precision-Recall Curve

## 6.2. Results Summary — Depression Dataset

The following table is to be completed with experimental results from the Depression pipeline upon completion of all training runs.

*Table 1: Depression Dataset — Comparative Performance Summary (Test Set)*

| Model               | Accuracy | Precision | Recall | F1-Score | ROC-AUC | PR-AUC |
|---------------------|----------|-----------|--------|----------|---------|--------|
| Naive Bayes         | 80.25%   | 80.25%    | 80.25% | 80.25%   | 88.14%  | 88.19% |
| SVM                 | 82.32%   | 82.33%    | 82.32% | 82.32%   | 90.29%  | 90.46% |
| Random Forest       | 77.52%   | 77.56%    | 77.52% | 77.51%   | 85.77%  | 85.73% |
| Logistic Regression | 81.36%   | 81.37%    | 81.36% | 81.36%   | 89.27%  | 89.44% |
| CNN                 | 83.05%   | 83.05%    | 83.05% | 83.05%   | 91.07%  | 91.30% |
| LSTM                | 83.50%   | 83.50%    | 83.50% | 83.50%   | 91.50%  | 91.67% |
| BiLSTM              | 83.53%   | 83.53%    | 83.53% | 83.53%   | 91.53%  | 91.69% |
| BERT                | 87.65%   | 87.65%    | 87.65% | 87.65%   | 94.87%  | 94.94% |

## 6.3. Results Summary — Suicide Dataset

The following table summarizes the test set performance of models on the Suicide dataset based on experimental results extracted from the notebook outputs:

*Table 2: Suicide Dataset — Comparative Performance Summary (Test Set):*

| Model               | Accuracy | Precision | Recall | F1-Score | ROC-AUC | PR-AUC |
|---------------------|----------|-----------|--------|----------|---------|--------|
| Naive Bayes         | 91.16%   | 91.53%    | 91.16% | 91.12%   | 97.72%  | 97.95% |
| SVM                 | 85.28%   | 85.30%    | 82.28% | 84.27%   | 92.63%  | 91.39% |
| Random Forest       | 81.96%   | 82.03%    | 81.96% | 81.97%   | 87.60%  | 86.20% |
| Logistic Regression | 86.93%   | 89.38%    | 85.01% | 87.14%   | 94.05%  | 94.38% |
| CNN                 | 94.59%   | 94.60%    | 94.59% | 94.59%   | 98.72%  | 98.80% |
| LSTM                | 94.24%   | 94.25%    | 94.24% | 94.24%   | 98.43%  | 98.47% |
| BiLSTM              | 93.97%   | 93.98%    | 93.97% | 93.97%   | 98.36%  | 98.36% |
| BERT                | 97.81%   | 97.81%    | 97.81% | 97.81%   | 99.64%  | 99.66% |

## 6.4. Classical ML Analysis

Classical machine learning models provide competitive baselines across both datasets while offering substantial advantages in training efficiency and interpretability. All four classical models on the Suicide dataset — Logistic Regression, Naïve Bayes, SVM, and Random Forest — are

implemented entirely from scratch without relying on pre-built library implementations, demonstrating the underlying mathematical foundations of each algorithm.

On the Depression dataset (1.6M tweets), SVM achieves the highest classical ML accuracy at 82.32% with an ROC-AUC of 90.29%, followed by Logistic Regression (81.36%), Naïve Bayes (80.25%), and Random Forest (77.52%). SVM's primal-formulation LinearSVC with balanced class weights and Platt-scaled probability calibration proves well-suited to the high-dimensional 100,000-feature TF-IDF space, completing training in just 1 minute. Naïve Bayes trains in under 30 seconds — the fastest model across both pipelines — demonstrating that the conditional independence assumption, while simplistic, remains remarkably effective for sparse TF-IDF representations. Random Forest, despite its 23-minute training time due to the OOB early stopping mechanism with incremental tree growth, trails the other classical models, suggesting that ensemble tree methods are less effective than linear models on this high-dimensional sparse feature space.

On the Suicide dataset (~242K Reddit posts), the performance ranking shifts. Naïve Bayes leads the classical models with 91.16% accuracy and the highest recall (90.95%) among the classical models, completing training in 2 minutes. Logistic Regression follows at 86.93% accuracy (3 minutes), outperforming SVM (85.28%, 1 minute) — a reversal of the Depression dataset ranking that suggests the full-batch gradient descent with L2 regularization captures the Suicide dataset's feature distributions more effectively than the hinge loss formulation. SVM's custom implementation with minority class upsampling addresses the class imbalance but achieves comparatively lower recall (82.93%), indicating that the linear decision boundary under hinge loss is less effective at identifying the positive class than NB's probabilistic framework. Random Forest's substantially lower performance (82.00%) and disproportionately long training time (114 minutes) can be attributed to its use of 13 hand-crafted features rather than the 20,000-dimensional TF-IDF representation used by the other models — the significantly reduced feature space limits discriminative capacity despite providing interpretability through explicit structural, sentiment, and keyword-based categories.

Across both datasets, the classical models collectively demonstrate that strong baselines can be achieved with training times ranging from 30 seconds to 3 minutes (excluding RF), making them practical for

resource-constrained or rapid-iteration scenarios where interpretability and deployment simplicity are prioritized over marginal accuracy gains.

## **6.5. Deep Learning Analysis**

The deep learning models demonstrate a consistent performance advantage over classical ML on both datasets, with the accuracy gap being particularly pronounced on the Suicide dataset. Training times increase relative to classical models but remain practical within Google Colab T4 GPU session limits.

On the Depression dataset, CNN (83.05%), LSTM (83.50%), and BiLSTM (83.57%) achieve closely clustered accuracies that exceed all classical models by 1–6 percentage points. BiLSTM's marginal lead over LSTM (83.57% vs. 83.50%) suggests that bidirectional processing provides only a slight benefit for the short tweet format where sequences are capped at 50 tokens. CNN trains fastest at 6 minutes, while LSTM (16 minutes) and BiLSTM (22 minutes) require more time due to the sequential nature of recurrent computation. All three models employ a hybrid dual-branch embedding strategy combining frozen GloVe Twitter 100-dimensional pre-trained vectors with trainable 200-dimensional domain-specific embeddings, producing a rich 300-dimensional representation per token. Training is regulated by three callbacks — early stopping, learning rate reduction on plateau, and model checkpointing — with a batch size of 512 optimized for the T4 GPU memory profile.

On the Suicide dataset, the deep learning models achieve substantially higher accuracies: CNN leads at 94.59%, followed by LSTM (94.24%) and BiLSTM (93.97%). The performance gap over classical ML widens to 3–12 percentage points, reflecting the ability of neural architectures to automatically learn discriminative feature representations from the longer Reddit posts (250 tokens) without manual feature engineering. CNN's leading performance suggests that local 5-gram patterns — specific phrases and word combinations associated with suicidal ideation — are the most discriminative features, and capturing these localized signals outweighs the benefit of modelling long-range sequential dependencies. The Suicide deep learning models use a simpler single-branch architecture with trainable 128-dimensional embeddings and set recurrent dropout to zero in LSTM and BiLSTM to enable CuDNN GPU acceleration, prioritizing training speed. All three Suicide deep learning models complete training in 4–5 minutes — faster than their Depression counterparts due to the smaller dataset size and simpler embedding architecture.

BERT fine-tuning achieves the highest overall performance on both datasets, confirming the advantage of bidirectional contextual embeddings and the self-attention mechanism for mental health text classification. On the Depression dataset, BERT reaches 87.65% accuracy with an ROC-AUC of 94.98% — a 4-percentage-point improvement over the best deep learning model (BiLSTM, 83.57%) and a 5-percentage-point improvement over the best classical model (SVM, 82.32%). On the Suicide dataset, BERT achieves 97.81% accuracy with near-perfect precision (97.81%) and recall (97.81%), outperforming CNN (94.59%) by nearly 4 percentage points. However, this performance comes at a significant computational cost: BERT requires 210 minutes on the Depression dataset and 270 minutes on the Suicide dataset — approximately 10–50× longer than the other deep learning models. The Suicide BERT model's longer training time despite the smaller dataset reflects the larger batch count resulting from the reduced batch size of 8 (necessitated by the 256-token sequence length) and on-the-fly tokenization.

## 6.6. Key Trade-offs

The results across both datasets reveal a clear accuracy–efficiency trade-off between model families. Classical ML models (LR, NB, SVM) deliver the fastest training times — as low as 30 seconds for Naïve Bayes on the Depression dataset and 1–3 minutes on the Suicide dataset — while achieving accuracies between 80–91%. Deep learning models (CNN, LSTM, BiLSTM) improve accuracy by 1–4 percentage points on both datasets at the cost of moderate training times (4–22 minutes), and BERT pushes accuracy further by an additional 4–5 percentage points but requires 210–270 minutes — roughly 10–50× longer than the other deep learning models. For rapid iteration, interpretability, or resource-constrained deployment, classical ML models offer the strongest value proposition; for production-grade classification where accuracy is paramount, BERT justifies the computational investment.

A second trade-off emerges between automatic and manual feature engineering. Classical ML models require explicit feature representations — TF-IDF vectorization for LR, NB, and SVM, or hand-crafted structural and sentiment features for Random Forest — while deep learning models learn representations automatically from embeddings. This distinction directly impacts scalability: TF-IDF-based models scale efficiently to the 1.6 million-sample Depression dataset through sparse matrix operations, whereas Random Forest's 13-feature engineering pipeline becomes a bottleneck (114 minutes on 242K samples) and limits discriminative capacity (82.00% accuracy versus

91.16% for NB on the same dataset). BERT's contextual embeddings eliminate feature engineering entirely but introduce a memory constraint — the 768-dimensional representations and transformer architecture require over 4 GB of GPU VRAM, necessitating batch size reductions (from 128 to 8) and on-the-fly tokenization to fit within the Colab T4 GPU's memory.

A third design consideration concerns embedding strategy across pipelines. GloVe Twitter 27B vectors are pre-trained on tweet data, making them domain-matched for the Depression pipeline (Sentiment140). The Suicide dataset, sourced from Reddit, exhibits longer posts and a distinct vocabulary, reducing the applicability of Twitter-derived embeddings. Trainable Keras embeddings were therefore used for the Suicide pipeline, allowing the model to learn task-specific representations from the Reddit corpus directly. While this introduces a difference in embedding strategy across pipelines, it reflects a deliberate domain-appropriate design decision rather than an inconsistency in the comparative framework.

Interpretability presents a third trade-off. Classical ML models offer direct transparency through TF-IDF feature weights (LR), class-conditional log-probabilities (NB), decision boundary margins (SVM), and explicit feature importance rankings (RF). This transparency is particularly valuable in the mental health domain where clinical trust and explainability are essential for responsible deployment. Deep learning and transformer models sacrifice this interpretability for performance — their learned representations, while more powerful, function as black boxes that require post-hoc explainability techniques such as attention visualization or LIME/SHAP analysis to understand their decision-making process.

## **6.7. Testing Strategy**

The testing and validation strategy relies on three complementary mechanisms to ensure correctness and reproducibility across all experiments.

First, runtime verification is embedded directly within each model script. Shape and data type assertions are printed at each stage of the pipeline — after data loading, preprocessing, feature extraction, and model output — to confirm dimensional consistency. Class distributions are verified after each train/test split to ensure stratification preserves the original label balance. For the Depression pipeline, the shared preprocessing artefacts (TF-IDF matrices, integer sequences, tokenizer,

and label encoder) are loaded by each downstream model, and their shapes are validated against the expected dimensions before training begins, ensuring all models operate on identical data representations.

Second, model-level validation is enforced through held-out data monitoring. All deep learning models reserve a validation split (5–10% of training data) for epoch-level monitoring, with early stopping preventing overfitting by restoring the best weights when validation loss ceases to improve. Classical ML models evaluate on both training and test sets, with the gap between the two providing a direct indicator of generalization. Fixed random seeds (42) and deterministic data splits ensure that all models are trained and evaluated on identical data partitions, eliminating data variation as a confounding factor in performance comparisons.

Third, evaluation outputs are cross-validated for consistency. All models generate predictions using sklearn's `classification_report`, providing standardized per-class Precision, Recall, and F1-Score as the ground truth for metric verification. Confusion matrices are visually inspected to confirm that raw counts are consistent with the reported metrics. For models producing probability outputs, ROC-AUC and Precision-Recall curves are plotted and visually compared against random-classifier baselines to verify discriminative capability. Full pipeline runs are conducted on the Google Colab T4 GPU environment to validate that all models execute within session time and memory constraints.

## 7. Conclusion

This project has successfully implemented and evaluated a comprehensive suite of eight models — four classical Machine Learning (LR, NB, SVM, RF), three Deep Learning (CNN, LSTM, BiLSTM), and one Transformer (BERT) — for mental health detection from social media text across two distinct domains. A unified experimental framework encompassing standardized preprocessing pipelines, identical stratified dataset splits, consistent evaluation metrics, and fixed random seeding ensures that observed performance differences between model families reflect genuine architectural differences rather than experimental artefacts.

On the Depression dataset (1.6M tweets), SVM achieved the highest classical ML accuracy at 82.32%, the deep learning models clustered closely between 83.05% (CNN) and 83.57% (BiLSTM), and BERT reached 87.65% — a 5-percentage-point improvement over the best classical model. On the Suicide dataset (~242K Reddit posts), the performance gap widened further: Naïve Bayes led the classical models at 91.16%, CNN achieved 94.59% among the deep learning models, and BERT attained 97.81% accuracy with near-perfect precision and recall. Across both datasets, BERT consistently delivers the highest accuracy, confirming the advantage of bidirectional contextual embeddings and self-attention for mental health text classification.

However, the results also demonstrate that accuracy alone does not determine practical suitability. Classical ML models train in seconds to minutes (0.5–11 minutes) while achieving 80–91% accuracy, offering full interpretability through feature weights and log-probabilities — a critical requirement in the mental health domain where clinical trust and explainability are essential. Deep learning models improve accuracy by 1–4 percentage points at moderate training cost (4–22 minutes), automatically learning discriminative features from embeddings without manual feature engineering. BERT's 4–5 percentage-point advantage over deep learning comes at a 10–50× increase in training time (210–270 minutes) and substantially higher GPU memory requirements. These trade-offs provide actionable guidance for deployment decisions: classical models for resource-constrained or interpretability-critical scenarios, deep learning for balanced performance-efficiency, and BERT when maximum accuracy justifies the computational investment.

The project contributes to the broader field of computational mental health research by providing a fully reproducible benchmark across two mental health domains, with all models — including from-scratch implementations of all four classical algorithms on the Suicide dataset — documented, evaluated, and compared under identical conditions. An interactive results dashboard has been developed to visualize and compare all model metrics, training curves, and confusion matrices in a unified interface.

### 7.1. Future Work

- **Explainability Analysis:** Apply LIME and SHAP to classical ML models and attention visualization to BiLSTM/BERT to provide deeper insight into the linguistic features driving predictions, enabling clinically interpretable explanations for model decisions.
- **Computational Cost Analysis:** Systematically document and compare training time, inference latency, GPU memory consumption, and carbon footprint across all model families and both datasets to provide practical deployment cost benchmarks.
- **Multi-class Extension:** Extend the binary classification framework to multi-class severity levels (e.g., mild, moderate, severe depression) for more granular mental health assessment, leveraging datasets with finer-grained annotations.
- **Domain-Specific Pre-training:** Evaluate specialized transformer variants such as MentalBERT — pre-trained on mental health corpora — to assess whether domain-specific pre-training yields further accuracy improvements over general-purpose bert-base-uncased.
- **Real-Time Deployment:** Package the best-performing models into a production-ready API with real-time inference capability, integrating the existing dashboard for live monitoring of model predictions and performance metrics.

*This project utilized Generative AI tools for limited assistance in brainstorming, documentation refinement, and coding support. All final implementation, analysis, and validation were carried out by the students.*

## 8. References

- [1] Z. Ding, Z. Wang, Y. Zhang, Y. Cao, Y. Liu, X. Shen, Y. Tian and J. Dai, "Trade-offs between machine learning and deep learning for mental illness detection on social media," *Scientific Reports*, vol. 15, 2025.
- [2] Y. Cao, J. Dai, Z. Wang, Y. Zhang, X. Shen, Y. Liu and Y. Tian, "Machine Learning Approaches for Mental Illness Detection on Social Media: A Systematic Review of Biases and Methodological Challenges," arXiv, 2025.
- [3] R. Safa, P. Bayat and L. Moghtader, "Automatic detection of depression symptoms in twitter using multimodal analysis," *Journal of Supercomputing*, vol. 78, pp. 4709-4744, 2022.
- [4] M. Kerasiotis, L. Ilias and D. Askounis, "Depression detection in social media posts using transformer-based models and auxiliary features," Springer, 2024.
- [5] B. G. Bokolo and Q. Liu, "Deep Learning-Based Depression Detection from Social Media: Comparative Evaluation of ML and Transformer Techniques," *Electronics*, vol. 12, p. 4396, 2023.
- [6] S. K. Sharma, A. I. Alutaibi, A. R. Khan, G. G. Tejani, F. Ahmad and S. J. Mousavirad, "Early detection of mental health disorders using machine learning models using behavioral and voice data analysis," *Scientific Reports*, vol. 15, 2025.
- [7] B. G. Bokolo and Q. Liu, "Advanced Comparative Analysis of Machine Learning and Transformer Models for Depression and Suicide Detection in Social Media Texts," *Electronics*, vol. 13, 2024.
- [8] R. Chandrasekaran, S. Kotaki and A. H. Nagaraja, "Detecting and tracking depression through temporal topic modeling of tweets: insights from a 180-day study," *npj Mental Health Research*, vol. 3, 2024.
- [9] L. Bendebane, Z. Laboudi, A. Saighi, H. Al-Tarawneh, A. Ouannas and G. Grassi, "A Multi-Class Deep Learning Approach for Early Detection of Depressive and Anxiety Disorders Using Twitter Data," *Algorithms*, vol. 16, 2023.
- [10] N. Hussain, A. Qasim, G. Mehak, M. Zain, G. Sidorov, A. Gelbukh and O. Kolesnikova, "Multi-Level Depression Severity Detection with Deep Transformers and Enhanced Machine Learning Techniques," *AI*, vol. 6, p. 157, 2025.
- [11] Z. Chen, R. Yang, S. Fu, N. Zong, H. Liu and M. Huang, "Detecting Reddit Users with Depression Using a Hybrid Neural Network SBERT-CNN," in *2023 IEEE 11th International Conference on Healthcare Informatics (ICHI 2023)*, 2023.
- [12] S. Zhou and M. Mohd, "Mental Health Safety and Depression Detection in Social Media Text Data: A Classification Approach Based on a Deep Learning Model," *IEEE Access*, vol. 13, pp. 63284 - 63297, 2025.

- [13] D. William and . D. Suhartono, "Text-based Depression Detection on Social Media Posts: A Systematic Literature Review," in *Proceedings of the 2020 International Conference on Computer Science and Computational Intelligence*, 2021.
- [14] D. Owen, J. Camacho-Collados and L. Espinosa-Anke, "Towards Preemptive Detection of Depression and Anxiety in Twitter," in *Social Media Mining for Health Applications (SMM4H)*, 2020.
- [15] N. M. Rum, N. A. Azura, F. A. Saharudin and A. Akbar, "Uncovering Depression on Social Media using BERT Model," *Journal of Advanced Research Design*, vol. 129, pp. 46-59, 2025.